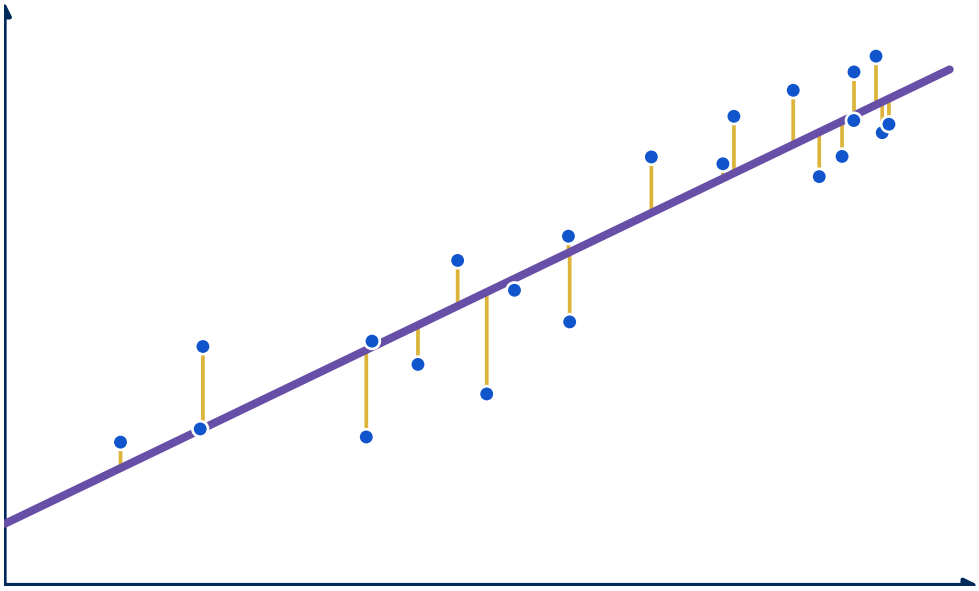


UNIT 01 · CHAPTER 4

Linear Regression

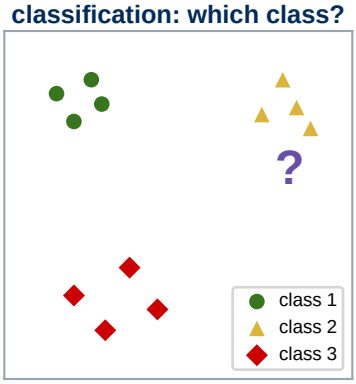
CSE 30124 · Introduction to Artificial Intelligence

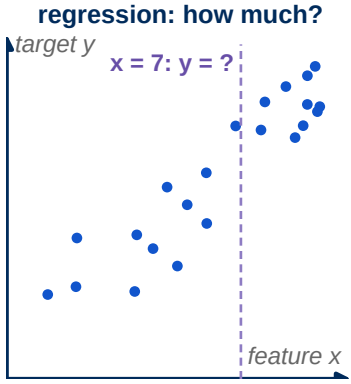
William Theisen · University of Notre Dame

Classification and regression

Our **previous models**, **k-Nearest Neighbors** and **Decision Trees**, were built for **classification**.

A **classification task** is one in which we try to assign a label (class) to a sample.





In **regression tasks**, we try and assign a continuous value to each sample.

Regression is another form of supervised learning.

Quick check (old exam question): is a regression task supervised or unsupervised learning?

Asking for a number

Both kNN and decision trees answer the same kind of question: which class does this sample belong to? Is this flower a setosa or a virginica, will John play tennis today or won't he. The answer is always **one label out of a short list** we decided on ahead of time.

Plenty of questions don't look like that. How many points will Notre Dame score on Saturday? There isn't a short list of answers to pick from, 27 and 28 are both perfectly reasonable guesses. We could turn it into a classification problem (0 to 10 points, 11 to 20 points, and so on) and it would technically work, but it throws away the fact that 21 is a lot closer to 20 than it is to 50. What we actually want is a model that hands back a number.

Linear regression

A **linear regression** is a technique to fit a line to our data, allowing us to make predictions.

Regression vocabulary

Unfortunately, because linear regression is so old, it **has a lot of vocab specific to it**:

[4.1] a line with one feature

$$\hat{y} = w_1 x_1 + w_0$$

- $\hat{y}$: the prediction (predicted target)
- w_1, w_0: the weights: slope and intercept (bias)
- x_1: the input feature

Note: statistics books write these weights as β_1 and β_0. We use w, and it's the letter we'll use for weights in every model from here on.

Running example. All the way through this chapter we'll be trying to answer one question: how many points will Notre Dame score, given how many yards it gains? We have the box score of every Notre Dame game from 2012 to 2022, 141 games in all. Our input feature x_1 is Notre Dame's total yards in a game, measured in hundreds of yards (so a 450-yard game is $x_1 = 4.5$), the target y is the points it scored and $\hat{y}$ is our model's guess. The slope w_1 is then "points per 100 yards", and the intercept w_0 is what the line predicts for a game with no yards at all.

Why a line?

A line is about the simplest model there is. It takes in a number and hands back a number, and it only has two knobs to turn: the **slope**, how much our prediction moves when the input moves, and the **intercept**, where the line starts. Two knobs also means we can look at a fitted line and say exactly what it learned, which is **more than we'll be able to say for most of the models later on**.

It's also **extremely old**. Legendre published the method of least squares in 1805. Gauss then claimed he'd been using it since 1795 to track the orbits of planets, he just hadn't gotten around to publishing it. The two of them argued about it for years, and astronomers, statisticians and economists have all been fitting lines (and inventing their own words for the parts) ever since.

Matrix notation

We can represent basically everything as **matrices** and **matrix operations**. Put a 1 at the front of every sample, $\mathbf{x}_i = [1, x_{i1}]$, and put the weights in one row, $\mathbf{w} = [w_0, w_1]$. Then a prediction is one dot product, and stacking the samples as the rows of X predicts all of them at once:

[4.2] every prediction at once

$$\hat{\mathbf{y}} = \mathbf{X}\,\mathbf{w}^{\mathsf{T}}$$

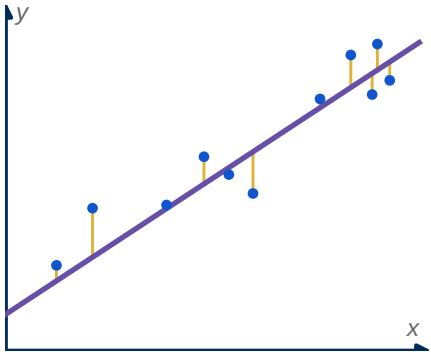
Multiple features

It's rare that we have datasets with only a single feature. The **iris dataset**, a toy dataset, has 4!

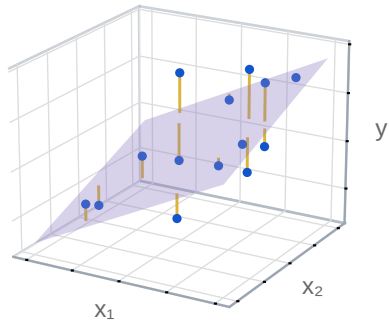
When we perform a **linear regression on** a dataset with **more than one feature**, we call this a multiple linear regression.

Hyperplanes

1 feature: a line



2 features: a plane



With one feature the model is a line, with two it's a **plane** and with more it's a **hyperplane**.

Quick check (old exam question): what would the equation look like for a multiple linear regression with three features?

A **hyperplane** is just a flat surface one dimension smaller than the space it lives in (a line in 2D, a plane in 3D, and past that we can't draw it but the math doesn't care). Each weight still means **how much ŷ moves when that one feature goes up by 1** and everything else stays put.

Two features: yards and turnovers

Back to football. A box score has a lot more in it than just yards, so let's give our model a second feature, x_2, Notre Dame's turnovers. Fitting to all 141 games we get:

two features, fit to 141 games

$$\hat{y} = 8.8\,x_1 - 2.7\,x_2 - 2.3$$

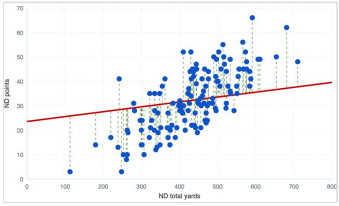
Each weight now answers a "holding everything else still" question. If yards stay the same, every turnover costs Notre Dame about 2.7 points. Notice that the yards weight barely moved (9.0 alone, 8.8 here), turnovers are telling the model something yards didn't. The line on its own explained 55% of the game-to-game variation in points, the plane explains 62%. (This "fraction of the variation explained" has a name, R^2, so the line has $R^2 = 0.55$ and the plane $R^2 = 0.62$.)

More features isn't always more information. If we swap turnovers for first downs instead, the first-downs weight comes out almost exactly 0 and the fit doesn't improve at all. Teams that gain a lot of yards also pick up a lot of first downs, so once yards is in the model first downs has nothing new to tell it.

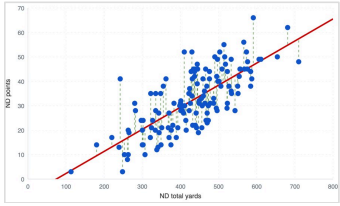
Fitting our line

How do we actually find the line that "fits" the best?

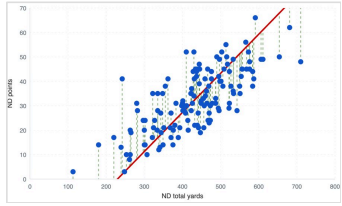
too flat: MSE 123



best: MSE 70



too steep: MSE 122



Live: open the football data in the visualizer, then drag the line yourself and watch the MSE.

Every one of these lines misses most of the games. The dotted segments are the misses, called **residuals**, $y_i - \hat{y}_i$. The line that's too flat under-predicts the big games and over-predicts the small ones, and the line that's too steep does the reverse. If we want to call one line the "best" we need a single number that sums up all 141 misses.

Optimization problems

We need some way to **measure how well our model fits** our data. We can call this a loss function.

For **linear regression** we often use **SSE** or MSE.

sum of squared errors

[4.3] $$SSE(w) = \sum_i (y_i - \hat{y}_i)^2$$

mean squared error

[4.4] $$MSE(w) = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

Why squared errors?

Why **square** the errors? If we just added up our residuals, a game we over-predicted by 10 points and a game we under-predicted by 10 points would cancel out, and a truly terrible line could end up with a total error of 0. Squaring makes every miss count against us.

It also means **big misses are punished much more than small ones** (being off by 20 points costs 400, being off by 2 costs only 4). Our line will try extremely hard not to be badly wrong about any single game, which is great until there's one truly bizarre game in the data, at which point the whole line gets dragged toward it. Unfortunately, college football has a lot of bizarre games.

The last reason is less exciting but probably the most important: a square is smooth, so we can take its derivative. We'll be doing exactly that in a page or two.

MSE is just SSE divided by the number of samples, n. Dividing everything by the same number doesn't move the bottom of the bowl, **so both pick the same line**. For our football line the MSE is about 70, which works out to a typical miss of about 8 points ($\sqrt{70} \approx 8.4$).

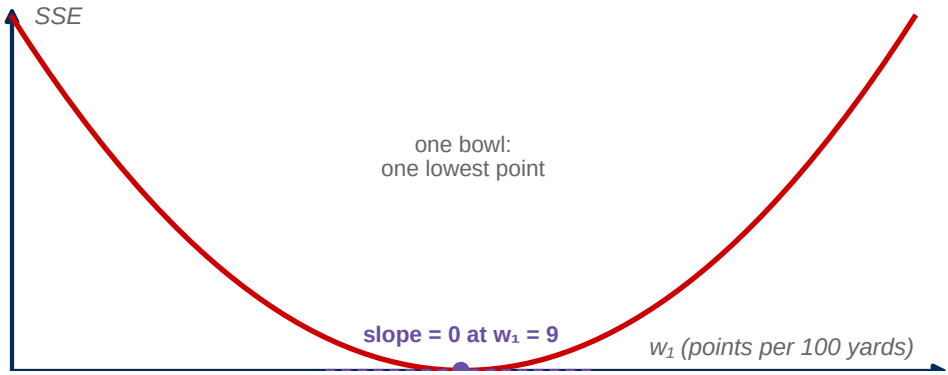
Finding our weights

So given a **loss function**, our **goal** is to minimize it.

When would a **loss function** be **minimized**? When the gradient is zero.

Visualizing loss

We can plot the loss function with respect to **w**:



This bowl is a parabola because SSE is a sum of squares of things that are linear in **w**. A bowl has **exactly one lowest point**, and at that point the slope is flat.

A shape like this is called **convex**, and that's extremely good news for us. There's no false bottom to get stuck in, so wherever the slope is zero is **the best answer, not just a pretty good one**. (The one catch is when $X^T X$ has no inverse, which we'll run into in a few pages. Then the bottom is a flat valley and lots of weights tie for best.)

One weight first

Before we do this with matrices, let's do it with a single weight. Take the five games from the worked example a couple of pages on, and leave out the intercept entirely so our line is just $\hat{y} = w_1 x$ (it has to go through 0, which is why the answer below comes out 7.8 instead of 9.0). Now the loss only has one knob, and we can take a regular derivative:

the loss, one weight

$$\mathrm{SSE}(w_1) = \sum_i (y_i - w_1 x_i)^2$$

↓ take the derivative and set it to 0

$$-2 \sum_i x_i (y_i - w_1 x_i) = 0$$

↓ solve for w_1

five games: 706 / 90

$$w_1 = \frac{\sum_i x_i y_i}{\sum_i x_i^2} \approx 7.8$$

With two weights we do exactly the same thing twice, once per weight, and both derivatives have to be zero at the same time. A list of derivatives like that, one per weight, is called the **gradient**. One more piece of notation for the next page: $\|v\|^2$ is the sum of the squares of the entries of v, so $\|y - X w^T\|^2$ is the SSE of every sample at once.

Ordinary least squares

We can **solve for the gradient equaling 0**:

SSE for every sample at once

$$L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}^T\|^2$$

↓ take the gradient and set it to 0

$$-2X^T(\mathbf{y} - X\mathbf{w}^T) = 0$$

↓ multiply out

the normal equations

$$X^T X \mathbf{w}^T = X^T \mathbf{y}$$

↓ multiply both sides by $(X^T X)^{-1}$

ordinary least squares [4.5]

$$\mathbf{w}^T = (X^T X)^{-1} X^T \mathbf{y}$$

This is called **Ordinary Least Squares (OLS)**.

Worked example: OLS by hand

Five games, yards rounded to the nearest hundred: x_1 is in hundreds of yards.

game	yards	x_1	y (points)
2019 Michigan	180	2	14
2012 Michigan State	300	3	20
2013 Purdue	400	4	31
2019 Boston College	501	5	40
2017 Temple	606	6	49

Give X a **column of 1s** for the intercept, then follow the **OLS** recipe.

the data, written on its side (transposed) to save room

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 3 & 4 & 5 & 6 \end{bmatrix} \qquad y^T = \begin{bmatrix} 14 & 20 & 31 & 40 & 49 \end{bmatrix}$$

two small products

$$X^TX = \begin{bmatrix} 5 & 20 \\ 20 & 90 \end{bmatrix} \qquad X^Ty = \begin{bmatrix} 154 \\ 706 \end{bmatrix}$$

2×2 inverse: swap, negate, divide by $5 \cdot 90 - 20 \cdot 20 = 50$

$$(X^TX)^{-1} = \frac{1}{50}\begin{bmatrix} 90 & -20 \\ -20 & 5 \end{bmatrix}$$

the weights

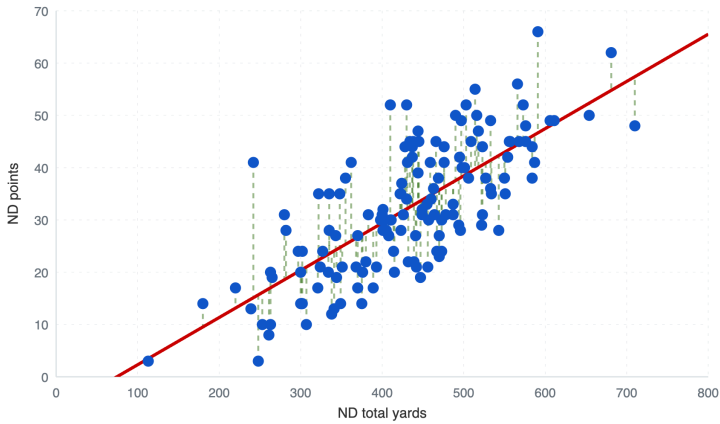
$$w^T = \frac{1}{50}\begin{bmatrix} 90 \cdot 154 - 20 \cdot 706 \\ -20 \cdot 154 + 5 \cdot 706 \end{bmatrix} = \begin{bmatrix} -5.2 \\ 9.0 \end{bmatrix} \tag{4.6}$$

Reading the weights

So $\hat{y} = 9.0\, x_1 - 5.2$: about 9 points per 100 yards. All 141 games give almost exactly the same slope (these five games happen to sit pretty close to the full-data line).

Quick check: in 2018 Notre Dame gained 550 yards against Stanford. What does this line predict, and what is the residual if Notre Dame scored 38?

Linear regression visualization



OLS on all 141 games gives $\hat{y} = 9.0\, x_1 - 6.7$. Be careful with the intercept though. It says a team that gains zero yards scores −6.7 points, which is obviously impossible. No game in our data is anywhere near zero yards, so the line was never asked to be right there.

In code: OLS

Here's the worked example in numpy. The `@` is matrix multiplication, and `np.column_stack` glues the column of 1s onto our X.

```python
import numpy as np

x = np.array([2, 3, 4, 5, 6])       # hundreds of yards
y = np.array([14, 20, 31, 40, 49])  # points

# the column of 1s, then x
X = np.column_stack([np.ones(len(x)), x])
w = np.linalg.inv(X.T @ X) @ X.T @ y
print(w)
```

OUTPUT

```
[-5.2  9. ]
```

numpy hands back a plain list, so w and w^T are the same thing to it. In practice we'd use `np.linalg.lstsq` instead of `inv` (it's faster and won't fall over when $X^T X$ is nearly singular), or just let sklearn do it (next page).

In code: sklearn

Here's the same fit on all 141 games, the way you'd actually do it (`nd_football.csv` is the box-score file):

```python
import pandas as pd
from sklearn.linear_model import LinearRegression

games = pd.read_csv("nd_football.csv")
X = games[["nd_total_yards"]] / 100   # hundreds
y = games["nd_points"]

model = LinearRegression().fit(X, y)
print(model.coef_.round(2), model.intercept_.round(2))
print(round(model.score(X, y), 2))    # R²
```

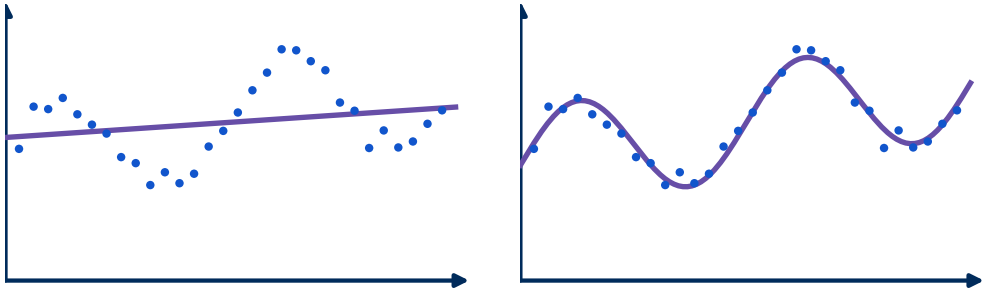
OUTPUT

```
[9.03] -6.73
0.55
```

Same 9.0 points per 100 yards and −6.7 intercept as before. `score` hands back R^2.

Representing curves

How could you use a **linear regression** to model a sine wave?



features: x

features: x and sin(1.25x)

The model only has to be linear in its **weights**, not in its inputs. We can hand it new features made from the old ones, like x^2 or $\sin(x)$, and it still fits them with the same equation:

[4.7] a straight line in new features

$$\hat{y} = w_0 + w_1 x + w_2 \sin(1.25x)$$

Building new features out of old ones like this is called **feature engineering**, and it's a lot more powerful than it looks. The catch is that someone has to know to try $\sin(1.25x)$ in the first place. When we get to SVMs we'll see a way to get huge numbers of these features almost for free, and neural networks go one step further and learn their own.

A possible issue

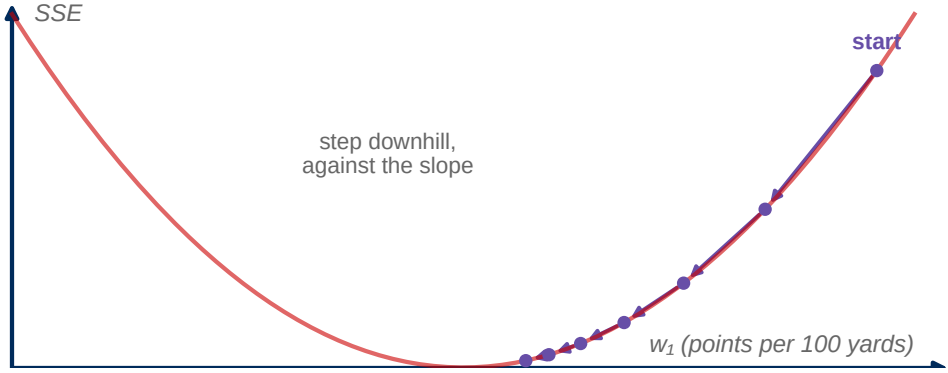
The **weight calculation relies on computing the inverse**, which can be **extremely computationally expensive** and **sometimes just literally impossible**.

With d features, $X^T X$ is a $(d+1) \times (d+1)$ matrix (one row and column per weight, intercept included), and inverting it takes on the order of d^3 steps, so a million features is hopeless. Worse, if one feature can be built from the others, $X^T X$ has no inverse at all.

Our football data walks straight into this. Total yards is just pass yards plus rush yards, so if we give the model all three as features, one column is the sum of the other two and $X^T X$ has no inverse. Technically ESPN's numbers disagree by 2 yards in a single game (the 2022 Gator Bowl), so the inverse does exist. It just isn't worth having. OLS hands back a weight of **−64** on total yards and about **+73** on both pass and rush yards (all per 100 yards). Taken at face value that says gaining yards costs you points, what's actually happening is the model latching onto one game's scorekeeping error. Funnily enough the predictions are still fine. An extra 100 passing yards adds −64 + 72 = 8 points and an extra 100 rushing yards adds −64 + 74 = 10, it's only the individual weights that have stopped meaning anything.

An iterative method

Instead of jumping straight to the bottom of the bowl, we can **walk downhill** a little at a time.



[4.8] one step of gradient descent

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

Here $\mathbf{w}$ is the whole list of weights, $\partial L / \partial \mathbf{w}$ is the gradient from a few pages back, and α is the **learning rate**, how big a step we take each time. Walking downhill fixes the cost problem, there's no inverse anywhere. Unfortunately, it doesn't fix the second problem. If $X^T X$ has no inverse the bottom of the bowl is a flat valley, and gradient descent will happily stop wherever in the valley it lands. We'll fix that one with regularization. (Gradient descent gets a whole chapter of its own later, so this is just a preview.)

In code: walking downhill

Gradient descent is only a few lines more. Start anywhere, compute the gradient, take a small step against it, repeat:

```python
import numpy as np

x = np.array([2, 3, 4, 5, 6])
y = np.array([14, 20, 31, 40, 49])
X = np.column_stack([np.ones(len(x)), x])

w = np.zeros(2)      # start anywhere
alpha = 0.005        # learning rate
for step in range(5000):
    grad = -2 * X.T @ (y - X @ w)   # gradient of SSE
    w = w - alpha * grad            # step downhill
print(w.round(2))
```

OUTPUT

```
[-5.2  9. ]
```

Same answer as the inverse, just found the slow way. Now try `alpha = 0.02`. Every number turns into `nan` within a few hundred steps, because a step that big jumps right over the bottom of the bowl and lands higher up the other side, and each step after that makes it worse. **Picking the learning rate is most of the work** in getting gradient descent to behave.

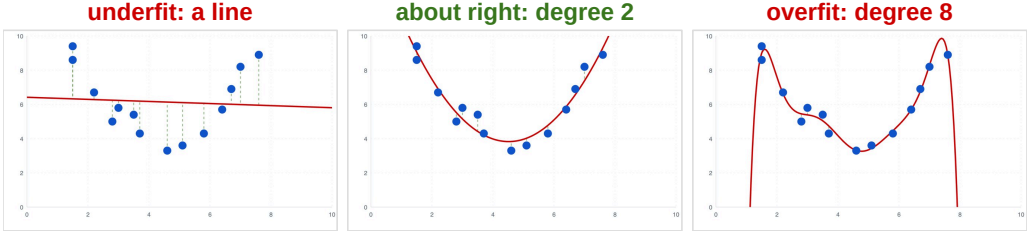
Maximum likelihood estimation

Maximum likelihood estimation (MLE) is a method of estimating the parameters of an assumed probability distribution, **given some observed data**.

Say we assume every game is the line plus some random noise, and that the noise follows a bell curve (a Gaussian). We can then ask which weights make the scores we actually saw the most likely. Work through the math and the log of that likelihood comes out to a constant minus $SSE / 2\sigma^2$, so **maximizing the likelihood is minimizing the SSE**. Least squares and maximum likelihood give the exact same line, linear regression has been doing MLE this whole time.

Overfitting

Unfortunately, **Linear Regressions** (a form of MLE), are **prone to overfitting**.



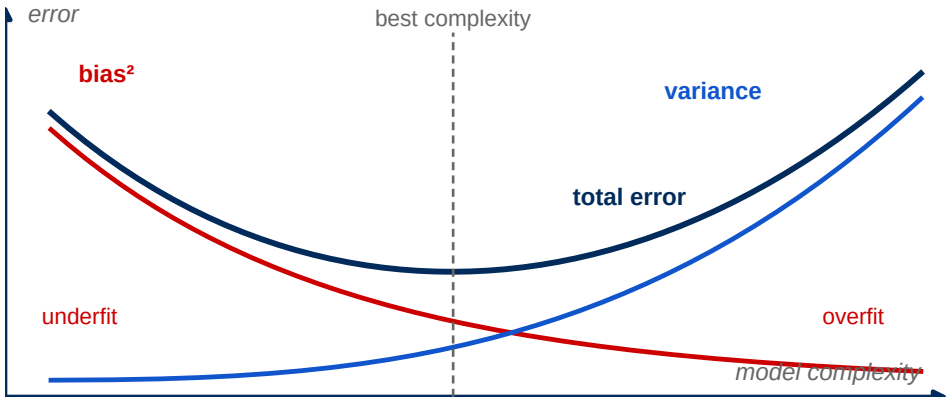
Live: open the degree-8 fit in the visualizer, then remove terms or drag a point and watch the curve swing.

Underfitting

Underfitting is when your model is "too weak" to capture all of the underlying patterns in the data.

A straight line through the wave on page 18 is underfit: no choice of slope and intercept can follow it.

Bias and variance



The **bias–variance tradeoff** describes the **relationship between** a **model's complexity**, the **accuracy** of its predictions, and how well it can make **predictions on unseen data** that were not used to train the model.

Two ways to be wrong

There are two different ways for a model to be wrong. **Bias** is how wrong the model is on average because it's too rigid to follow the pattern. The straight line through the U-shaped points on page 22 is high-bias, give it a different sample of points and it will miss the curve in exactly the same way. This is underfitting.

Variance is how much the model changes when the training data changes. The degree-8 curve is high-variance, move a single point and the swings near the edges flip around completely. This is overfitting.

Adding complexity lowers bias and raises variance, so the best model sits somewhere in the middle, at the bottom of the total-error curve. Unfortunately, **training error can't tell us where that is**. The degree-8 curve has the lowest training error of the three! The only way to know we've gone too far is to check the model on data it hasn't seen.

Practice: an old exam question

x_1	x_2	y
1	1	9
2	2	25
3	3	50
4	4	84
5	5	127

Given the dataset above, answer the following questions **(Old exam question)**:

A. Would a **purely linear model** exhibit **high bias** here or **high variance**? Would it be **overfit** or **underfit**? Explain your reasoning.

B. Design a regression equation that would be able to approximate the dataset.

C. What role does the **SSE** play in **linear regression**? Why does it matter that our resulting parameter space is convex?

D. Look at the x_1 and x_2 columns. What goes wrong if we fit $\hat{y} = w_0 + w_1 x_1 + w_2 x_2$ with **OLS**?

Regularization: L_1 and L_2

To help **prevent overfitting** when using **linear regressions**, we can perform a technique called **regularization**.

The goal of **regularization** is to **encourage models to learn the broader patterns** within the data rather than memorizing it.

L_2 regularization: Ridge

L2 regularization (called **ridge regularization**) **encourages small (but nonzero) weights**.

L_1 regularization: LASSO

L1 regularization (also called **LASSO**) leads to weights of exactly zero and **sparse models**.

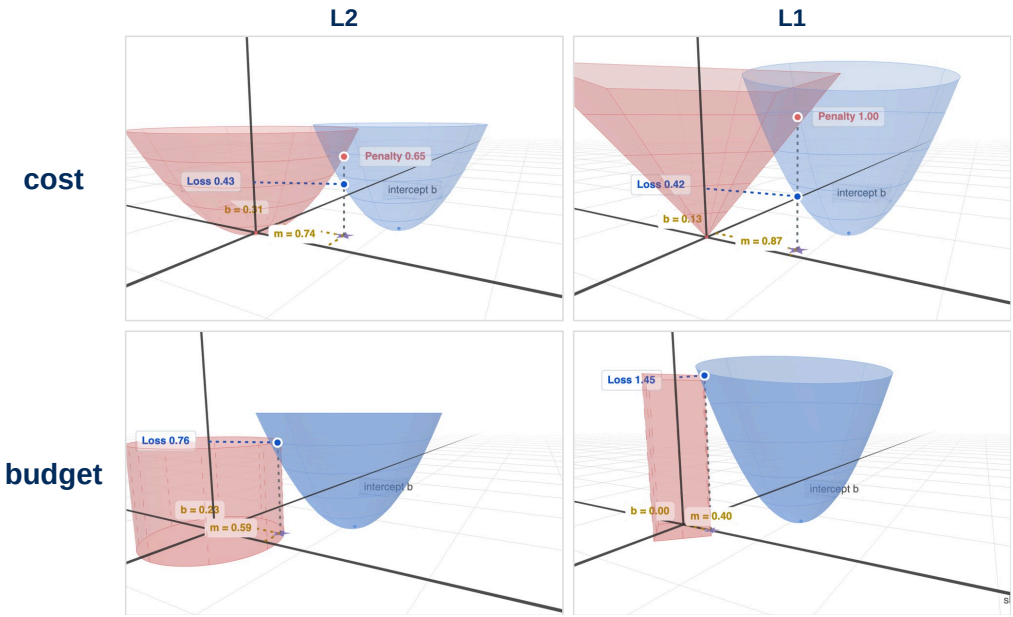
Why small weights?

Why would big weights mean overfitting? Look back at the degree-8 curve through the U-shaped points. To swing through every point it needs weights in the hundreds and thousands (the biggest is about 1,023), each one mostly cancelling out the one next to it. The straight line's biggest weight is about 6 and the degree-2 curve's is about 16. A model that needs **enormous, finely balanced weights** is almost always fitting the noise.

Formalizations of regularization

There are two ways to formalize regularization:

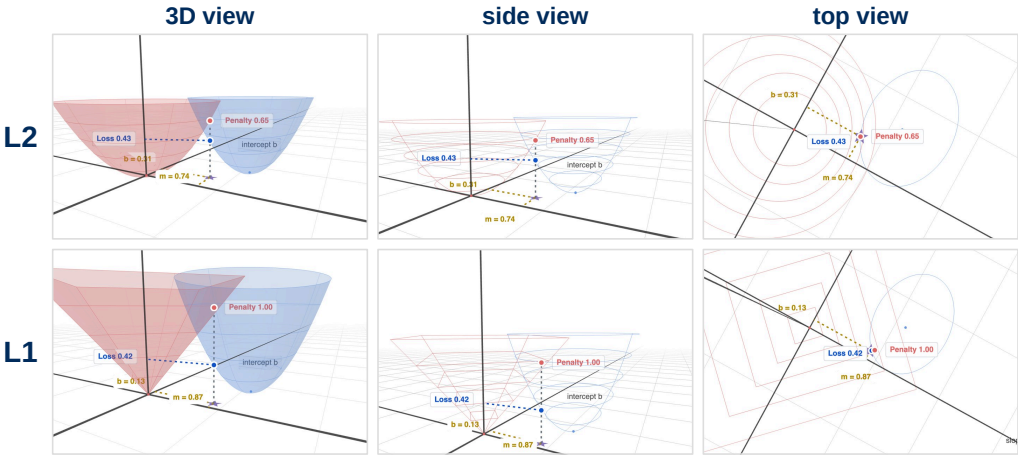
- As a cost (penalty) placed on the **weights**
- As a budget (constraint) for the **weights**



Live: open this view in the visualizer, then change the penalty, λ or t yourself.

Regularization as a cost

When we formalize **regularization** as a **cost**, we modify the **loss function** such that increasing the weights incurs a "cost".



Live: open this view in the visualizer, then change the penalty, λ or t yourself.

We use an argument called **λ** to determine how heavily the **cost** is weighed. A larger **λ** increases the **cost**.

L2 as a cost

$$[4.9] \quad L(w) = SSE(w) + \lambda \sum_j w_j^2$$

L1 as a cost

$$[4.10] \quad L(w) = SSE(w) + \lambda \sum_j |w_j|$$

Ridge fixes the inverse

Remember the possible issue from earlier, $X^T X$ not having an inverse? Ridge fixes it almost by accident. Take the gradient of the ridge loss, set it to zero like we did for OLS, and the only change is a λ added down the diagonal:

[4.11] ridge, solved in one step

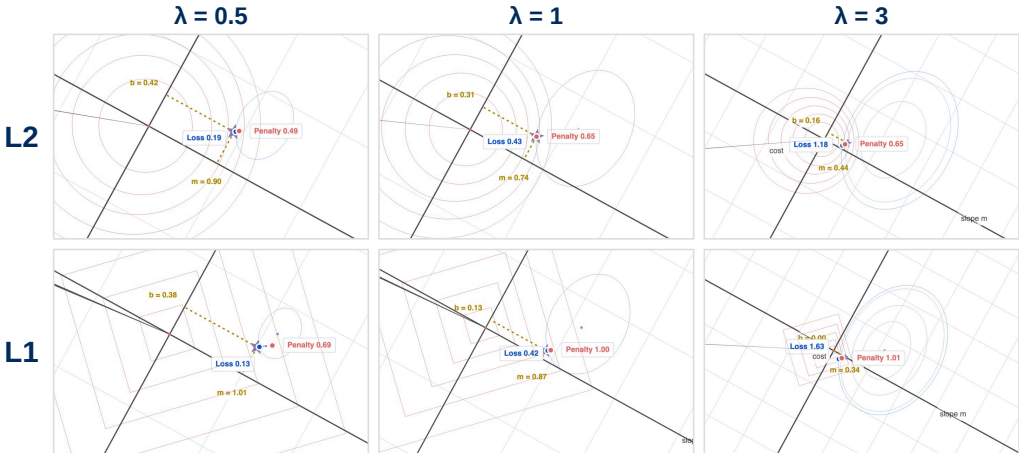
$$w^T = (X^T X + \lambda I)^{-1} X^T y$$

Adding any positive λ to the diagonal **guarantees the inverse exists**, so the flat valley turns back into a bowl with one bottom. In practice we usually leave the intercept w_0 out of the penalty (there's no reason to push the average score toward 0), so the top-left entry of I becomes a 0.

Back to the pass, rush and total yards. Plain OLS gave us weights of −64, +72 and +74. Ridge with $\lambda = 1$ gives **+5.9** on total yards, **+2.0** on pass yards and **+3.9** on rush yards. The predictions barely change (100 passing yards is still worth about 8 points and 100 rushing yards about 10), but now every weight is positive and none of them are absurd.

Regularization as a cost, increasing λ

A larger λ increases the **cost** of our weights, typically resulting in the optimum being pulled closer to the origin.



Live: open this view in the visualizer, then change the penalty, λ or t yourself.

Regularization as a budget

When we formalize **regularization** as a **budget**, we constrain our **loss function** such that the weights chosen must remain within "budget".

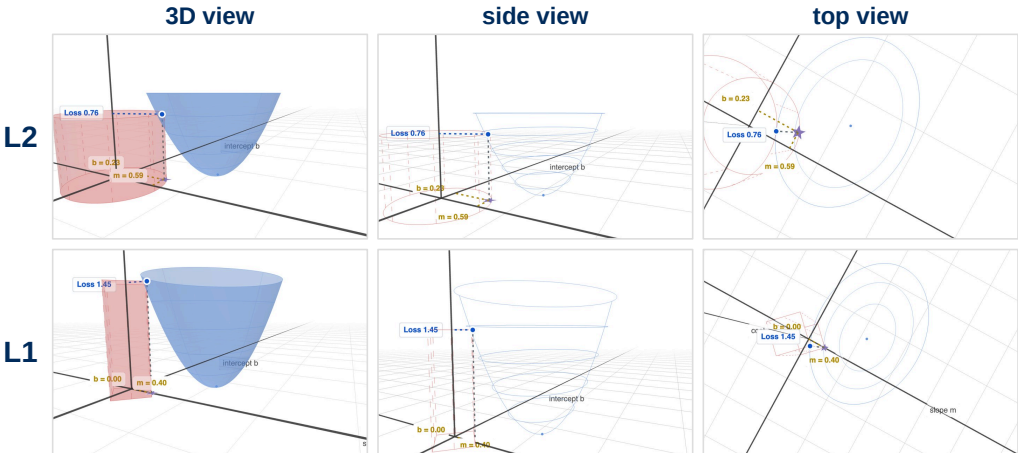
L2 as a budget

$$\min_{w} \; SSE(w) \quad \text{subject to} \quad \sum_{j} w_j^2 \leq t$$

L1 as a budget

$$\min_{w} \; SSE(w) \quad \text{subject to} \quad \sum_{j} |w_j| \leq t$$

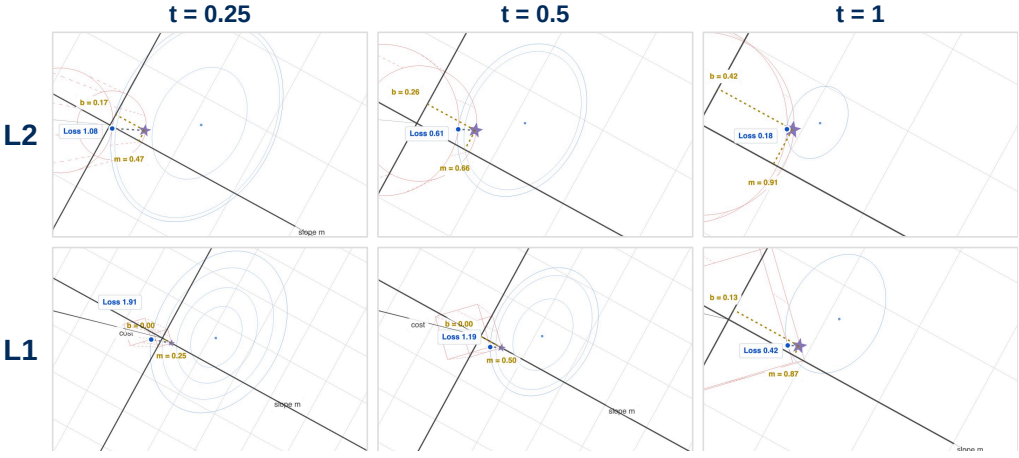
Regularization as a budget



Live: open this view in the visualizer, then change the penalty, λ or t yourself.

Increasing t

A larger t increases the **budget** for our weights, typically resulting in the optimum being closer to the unconstrained optimum.



Live: open this view in the visualizer, then change the penalty, λ or t yourself.

Corners and budgets

Look at the L1 row of the cost pictures. At λ = 3 the star lands exactly on one of the axes, so one of the two weights is exactly 0. That's what "sparse" means. The budget pictures show why it happens: the L1 budget is a diamond, and the loss bowl almost always touches a diamond first at one of its corners, which sit right on the axes. The L2 budget is a circle with no corners, so its star gets pulled in close to an axis but never quite lands on it.

The cost and the budget aren't really two different ideas, they're the same problem written two ways. For every budget t there's some λ that puts the star in exactly the same spot (showing it takes Lagrange multipliers, which we'll see again with SVMs). **A big λ is a small budget**, and a small λ is a big one.

LASSO picks the features

A weight of exactly 0 means that feature is out of the model entirely, so L1 is doing **feature selection** for us for free. Let's give it something to select. Here are seven things from Notre Dame's box score: total yards, first downs, turnovers, penalties, third-down conversions, time of possession and number of plays.

Plain OLS uses all seven and explains 64% of the variation in points. LASSO with a moderate λ keeps exactly two of them, **total yards** (about 7.9 points per 100 yards) and **turnovers** (about −1.9 points each), and still explains 61%. The other five weights are exactly 0. Most of what a box score tells you about points, it turns out, is how far you moved the ball and how often you gave it away.

One practical detail: before using LASSO we rescale every feature to the same spread. Otherwise a feature measured in big units (yards) and one measured in small units (turnovers) get penalized very differently for the same amount of usefulness.

In code: ridge

sklearn has both. One annoying detail: sklearn calls λ `alpha`, which has nothing to do with the learning rate from gradient descent.

```
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge

games = pd.read_csv("nd_football.csv")
cols = ["nd_total_yards", "nd_pass_yards",
        "nd_rush_yards"]
X = games[cols] / 100
y = games["nd_points"]

ols = LinearRegression().fit(X, y)
ridge = Ridge(alpha=1).fit(X, y)   # alpha is λ here
print(ols.coef_.round(1))
print(ridge.coef_.round(1))
```

OUTPUT

```
[-64.4  72.4  74.2]
[5.9 2.  3.9]
```

Same predictions as OLS, sensible weights, exactly as on the ridge page.

In code: LASSO

LASSO needs one extra step first, rescaling every feature to the same spread:

```
import pandas as pd
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler

games = pd.read_csv("nd_football.csv")
stats = ["nd_total_yards", "nd_first_downs",
         "nd_turnovers", "nd_penalties",
         "nd_third_down_conv", "nd_possession_min",
         "nd_plays"]
X = StandardScaler().fit_transform(games[stats])
y = games["nd_points"]

lasso = Lasso(alpha=1).fit(X, y)
for name, w in zip(stats, lasso.coef_):
    print(f"{name:20s} {w + 0:6.2f}")
```

OUTPUT

```
nd_total_yards         8.16
nd_first_downs         0.00
nd_turnovers          -2.29
nd_penalties           0.00
nd_third_down_conv     0.00
nd_possession_min      0.00
nd_plays               0.00
```

These LASSO weights are per standard deviation (that's what `StandardScaler` did), so compare them to each other rather than to the 9 points per 100 yards from before. Five of the seven are exactly 0.

MAP estimation

L1 and **L2 regularization** are a form of **MAP Estimation**.

MAP stands for Maximum A Posteriori which is a **Bayesian** approach for **applying a prior to our weights**.

MLE only asks one question: which weights make the data we saw the most likely? MAP asks that too, but it also asks which weights we thought were likely before we saw any data at all. That second part is the **prior**. If we believe weights are probably small and close to zero (a Gaussian prior), the math turns that belief into exactly the $\lambda \Sigma w_j^2$ term from ridge. A sharper prior that piles up right at zero (a Laplace prior) turns into the $\lambda \Sigma |w_j|$ term from LASSO instead. So **regularization is really just us telling the model what we expected ahead of time**.

Review

Complete the review: **L1 regularization** penalizes the absolute values of the weights. **L2 regularization** penalizes the squared weights. In a penalty formulation, increasing **lambda** makes the penalty stronger. In a budget formulation, increasing the budget makes the constraint looser.

Matching: regression and regularization

F **Ordinary Least Squares**	**A.** Penalizes the squared size of the weights, encouraging small weights
G **Maximum Likelihood Estimation**	**B.** Controls how heavily the regularization cost is weighed
J **MAP Estimation**	**C.** The budget the weights must remain within
A **L2 (Ridge)**	**D.** The model memorizes noise in the training data instead of the general pattern
I **L1 (LASSO)**	**E.** The relationship between model complexity, training accuracy, and accuracy on unseen data
B λ	**F.** Solving for the weights by setting the gradient of the loss equal to 0
C t	**G.** Estimating the parameters that make the observed data most likely
H **Underfitting**	**H.** The model is too weak to capture the underlying patterns in the data
D **Overfitting**	**I.** Penalizes the absolute size of the weights, driving some to exactly 0
E **Bias–variance tradeoff**	**J.** A Bayesian approach for applying a prior to our weights

Quick-check answers

p. 2: Supervised: every training sample comes with the true target value, so we can measure how far off each prediction is.

p. 7: $\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_0$: one weight per feature, plus the intercept.

p. 15: $x_1 = 5.5$, so $\hat{y} = 9.0 \times 5.5 - 5.2 = 44.3$ points. The residual is $y - \hat{y} = 38 - 44.3 = -6.3$: Notre Dame scored about 6 points fewer than its yards suggest.

p. 25: High bias, underfit. The jumps in y are 16, 25, 34, 43: they grow by 9 every step, so y is quadratic and no straight line can follow it.

p. 25: Add a squared feature: $\hat{y} = w_0 + w_1 x_1 + w_2 x_1^2$. In fact $y = 4.5x_1^2 + 2.5x_1 + 2$ hits all five points exactly.

p. 25: SSE is the loss we minimize to choose the weights. Because it's convex (strictly, it's the loss that's convex, not the parameter space) there are no false bottoms: any point where the gradient is zero is a best answer, so setting the gradient to zero (or walking downhill) finds the best weights, not just a local dip.

p. 25: The two columns are identical, so $X^T X$ has no inverse. Any pair of weights with the same $w_1 + w_2$ fits equally well, so there's no single answer. Drop one of the two columns (or regularize).